KU LEUVEN

Chapter 6. Predictive Analytics: Artificial Neural Networks

Data Science for Business

D0I74A

Prof. dr. Wouter Verbeke

Prof. dr. Bart Baesens



Course schedule

More on ANNs in Chapters 7, 9, 11, 12, 13

BLOCK2			
6	26/10/2026	Chapter 6. Predictive Analytics - Artificial Neural Networks	
6	27/10/2026	No class, video-lecture: Chapter 7. Predictive Analytics - Forecasting	
7	02/11/2026	**No class, university holiday**	
7	03/11/2026	Chapter 9. Prescriptive Analytics - Causal Machine Learning	
8	09/11/2026	No class, video-lecture: Chapter 8. Descriptive Analytics	
8	10/11/2026	Chapter 9. Prescriptive Analytics - Causal Machine Learning	**Report 1 deadline**
9	16/11/2026	Chapter 10. Prescriptive Analytics - Profit-driven Analytics	
9	17/11/2026	Guest Lecture #1: Toon Vanderschueren, Ph.D., Causal ML @Telenet	**Presence expected**
	17/11/2026	Assignment 2	**Report 2 > 14/12**
10	23/11/2026	Chapter 11. Prescriptive Analytics - Multi-armed Bandits & Reinforcement Learning	
10	24/11/2026	Guest Lecture #2: Dr. Véronique Van Vlasselaer, Ph.D., Business Analytics @SAS	**Presence expected**
BLOCK3			
11	30/11/2026	Chapter 12. Natural Language Processing	
11	01/12/2026	Chapter 12. Natural Language Processing	
12	07/12/2026	Chapter 13. Large Language Models & AI Agents	
12	08/12/2026	Chapter 13. Large Language Models & AI Agents	
13	14/12/2026	Feedback Assignment 2 Sample Exam Questions	**Report 2 deadline**
13	15/12/2026	Guest Lecture #3: To be confirmed	**Presence expected**

Warning

- Artificial Neural Networks (and Deep Learning):
  - Concern a broad *family* of related methods
  - Which have been widely studied
  - For many years
  - Resulting in a vast body of literature
  - Entailing lots of *complexity*

- Therefore, this chapter:
  - Only introduces some of the very basic – yet fundamental – concepts
  - Which allow you to enter and further explore this fascinating field
  - And allows you to use ANNs for business decision-making

Outline

1. The neuron model
2. Architecture
3. Training
4. Learning vs. Overfitting
5. Demo

Learning objectives

- At the end of this chapter, students should be able to:
  - **Explain** artificial neural networks and the **relationship** with other modeling approaches
  - Discuss on the **architecture** of ANNs and the various choices that have to be made
  - Explain **gradient descent learning** and the **backpropagation** algorithm, including **hyperparameters** that govern the algorithm and their impact on the learning process
  - Explain the **bias – variance trade-off** with ANNs and potential solutions
  - Discuss the **challenges** related to developing ANNs, as well as the **benefits** and **drawbacks** of using ANNs for business decision making

1. The neuron model

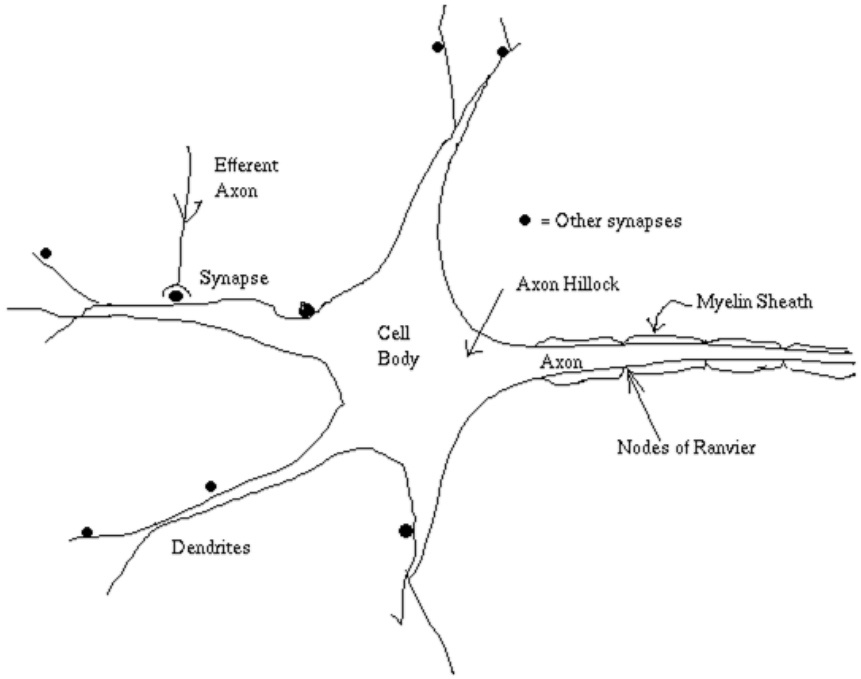
Background: two views

Background: two views

1. Biological view
   - **ANN = mathematical representation** of the workings of the human brain

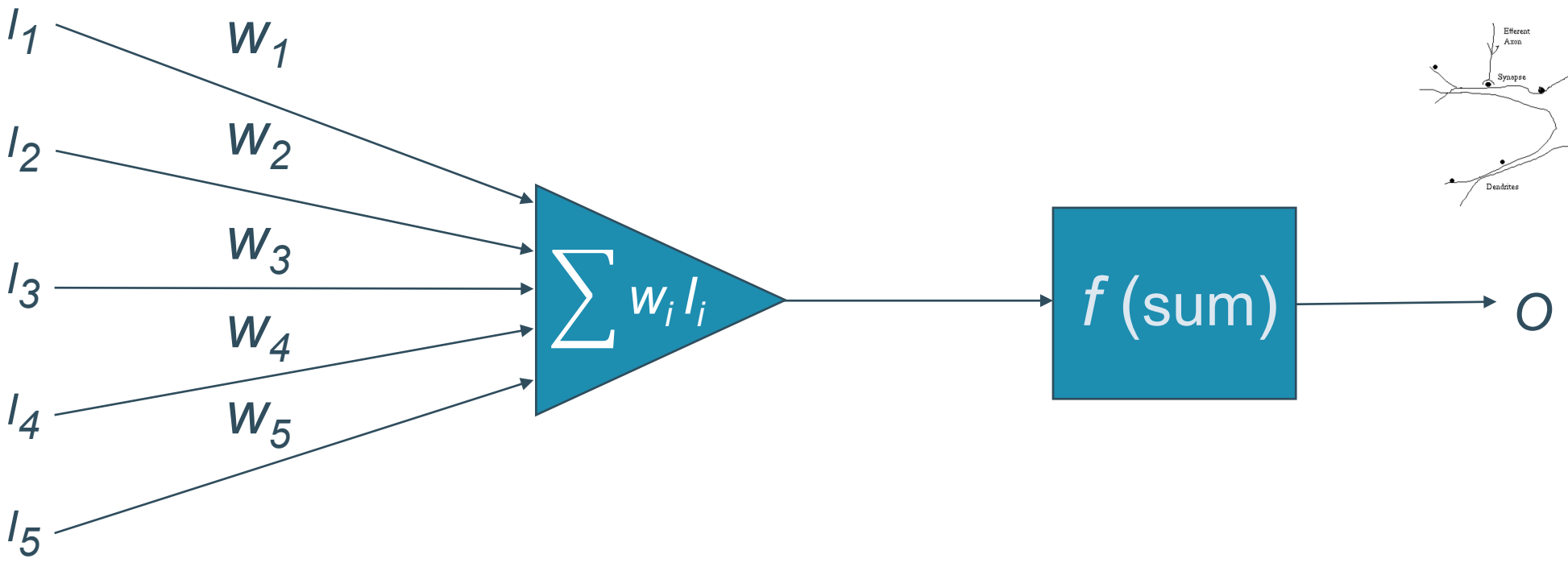
   - Basic building block of brain: brain cell or neuron
     - Dendrites provide inputs
     - Axons send outputs
     - Nucleus or cell body transforms inputs into outputs

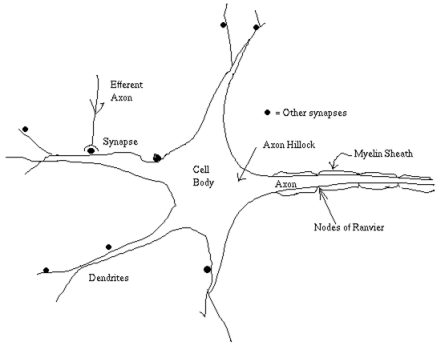
   - To mimic the workings of a brain
     → We can model the workings of a neuron
     → The **neuron model**



Background: two views

The neuron model
= mathematical model of a neuron
= basic building block of ANNs



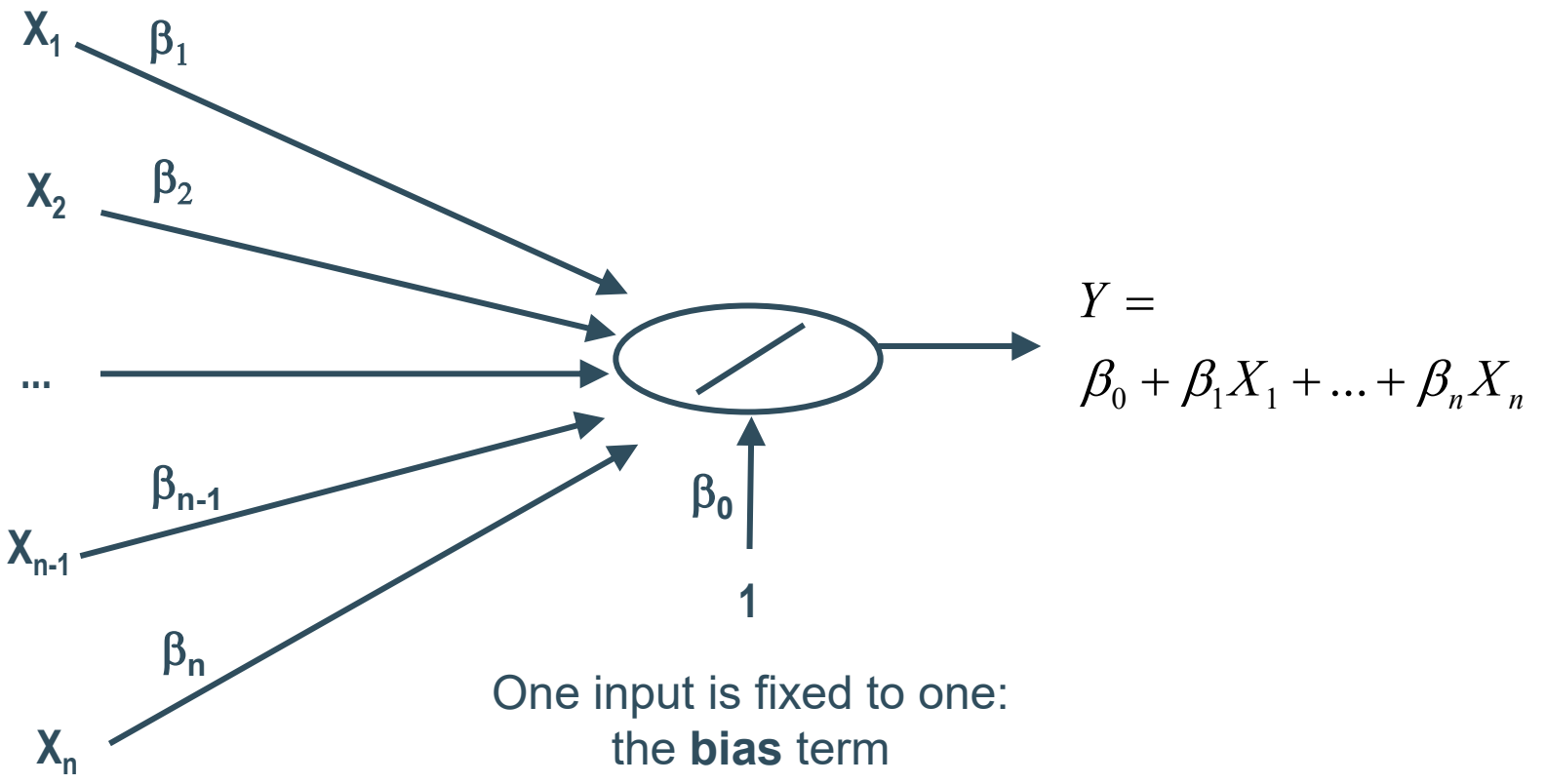


inputs **weights** **sum** = $\sum w_i I_i$ (combination function) ***f* : activation function** (aka squashing or transfer function) **output**

Background: two views

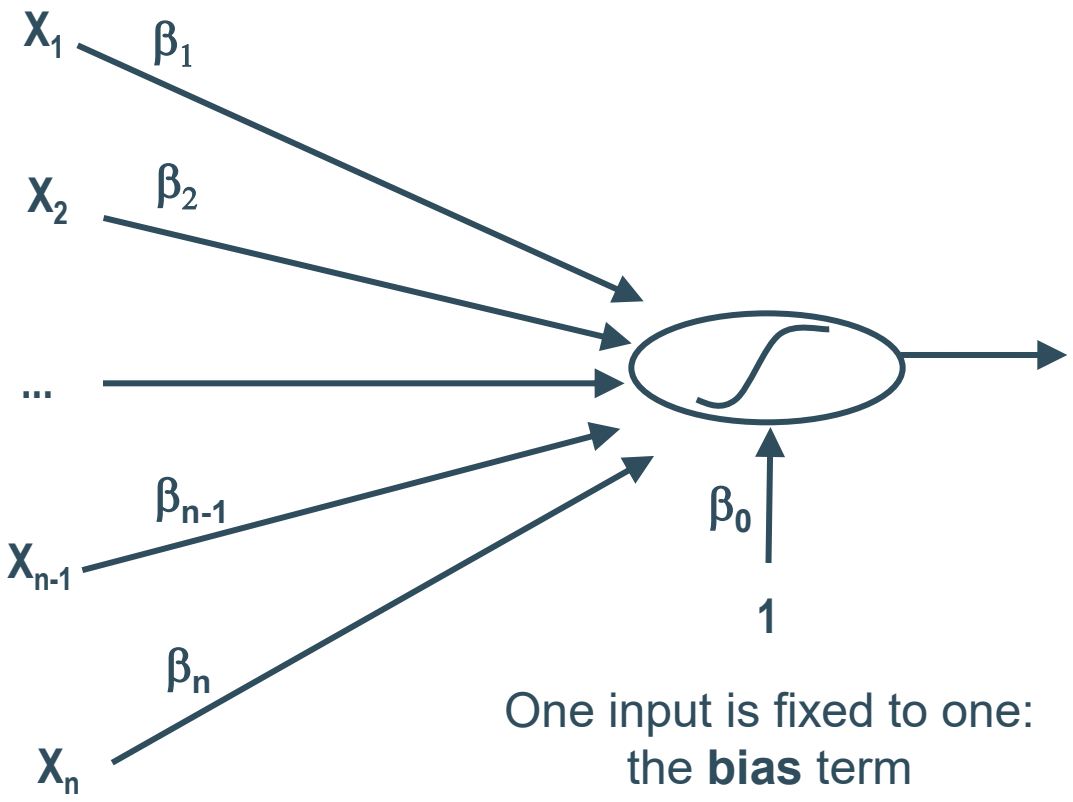
1. Biological view
2. Statistical view
   - Neural networks are **generalizations of existing statistical models**
   - For example, of linear and logistic regression models

Neural network representation: Linear Regression



Neural network representation: Logistic Regression

Generalization?



One input is fixed to one:
the **bias** term

Neural network:

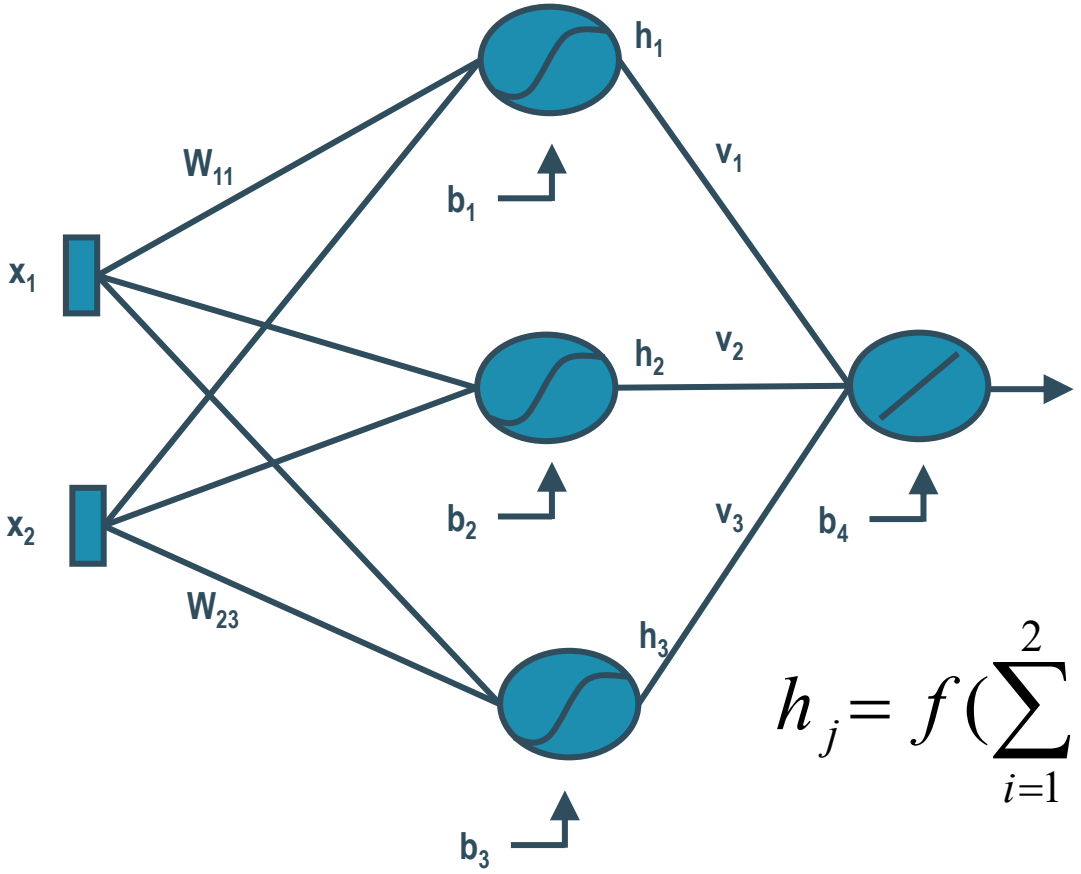
- Can be a **linear regression** model
- Can be a **logistic regression** model (or cloglog, probit, …)
- Can be ... **anything** (?!) else

$$P(Y \mid X_1,...,X_n) = \frac{1}{1+e^{-(\beta_0+\beta_1 X_1+...+\beta_n X_n)}}$$

Multilayer Perceptron (MLP)

Combining multiple neurons greatly extends the ***modeling flexibility***, allowing to fit highly **complex** models

Generalization?



$$z = \sum_{j=1}^{3} v_j h_j + b_4$$

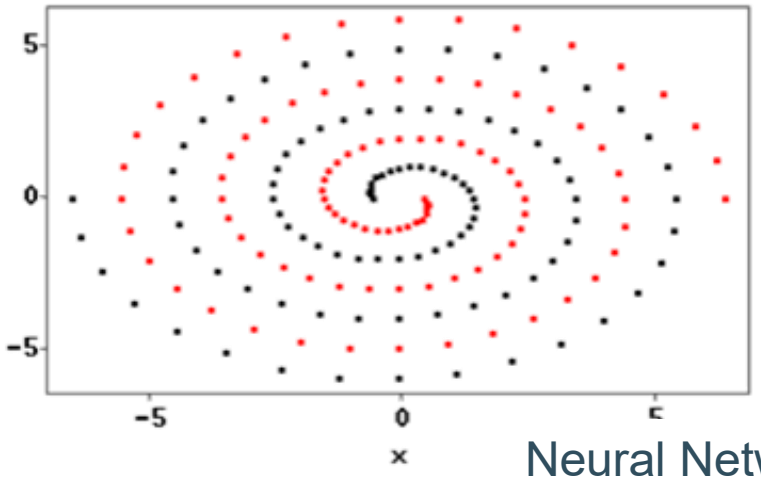
$$h_j = f\left(\sum_{i=1}^{2} x_i w_{ij} + b_j\right)$$

Multilayer Perceptron (MLP)

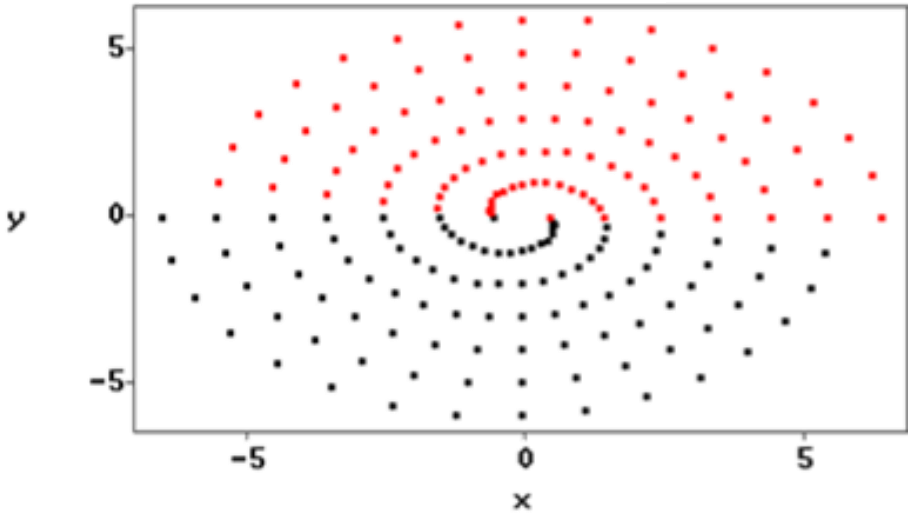
Combining multiple neurons greatly extends the ***modeling flexibility***, allowing to fit highly **complex** models

Classification problem: classify instance (x,y) as red or black

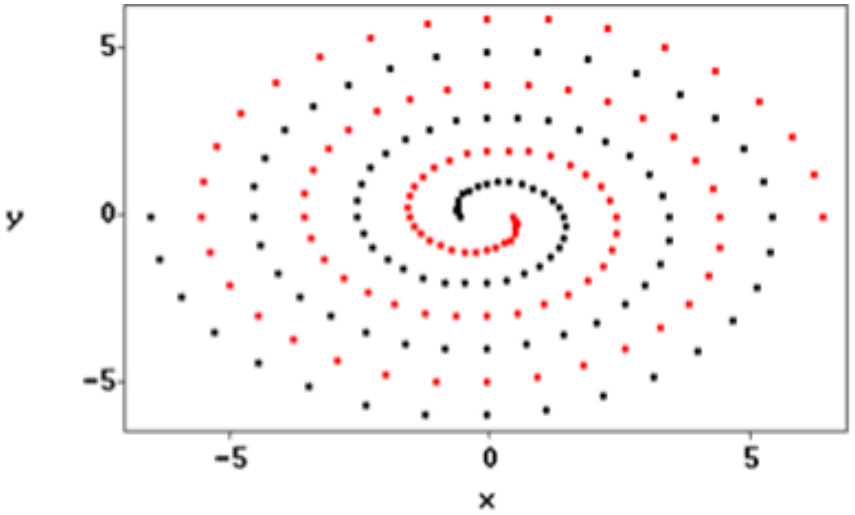
- Nonlinear problem!



Logistic Regression



Neural Network (40 hidden neurons)



Terminology: neural nets vs. statistics

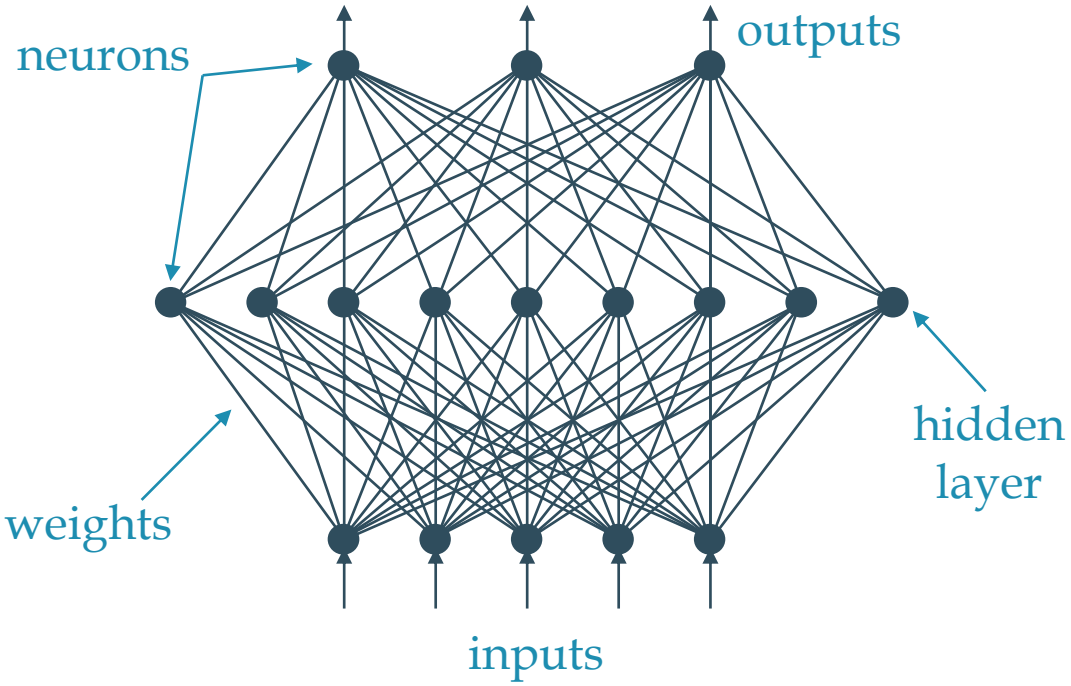
Neural Networks	Statistics
Learning, Training	Fitting, Optimizing, Estimating
Bias	Intercept
Weight	Parameter estimate, Coefficient
Epoch	Iteration, Step
Pruning, Input selection	Model reduction, Variable selection
Architecture	Model

Architecture?

2. Architecture

Architecture

- **Architecture?**
  - Series of **choices**
  - Regarding the **type**(s) of neurons
  - And how they are **combined** in a network



- **Multilayer Perceptron (MLP)**
  - Type of architecture
  - Neurons are organized in **layers**
    - Input layer
    - Hidden layer(s)
    - Output layer
  - Note: if k inputs, h hidden neurons (one hidden layer), 1 output, then (k+1)h+h+1 weight parameters for **fully connected MLP**

Multilayer Perceptron

- MLPs are the most common type of neural network for supervised prediction in decision support systems, for example, in credit scoring, churn prediction, response modeling, etc.

- Multiple **hidden layers** possible
  - Universal approximation property with 1 hidden layer
  - **No feedback connections** (recurrent neural networks, RNNs)
  - **Skip layer** neural network: direct connections between inputs and output

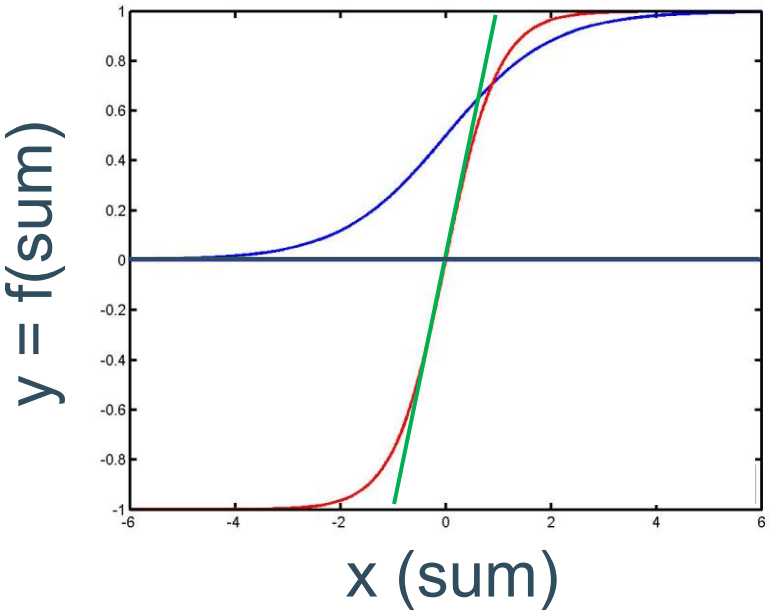
- Every neuron has a **bias input** (intercept in linear regression)

- **Combination functions**
  - Linear: linear combination of incoming values and weights (default)
  - Additive: adds all incoming values without using weights or biases
  - Equal slopes: same weights for each neuron per layer with different biases

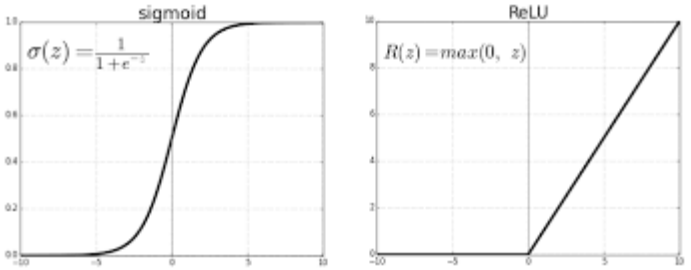
- **Activation functions**

Activation function f

- Sigmoid (logistic)
  - $f(sum)=1/(1+e^{(-sum)})$
  - between 0 and 1
- Hyperbolic tangent (tanh)
  - $f(sum)=(e^{(sum)}-e^{(-sum)})/(e^{(sum)}+e^{(-sum)})$
  - between -1 and 1
  - same as 2*logistic(2*sum)-1
  - default for hidden units
- Linear
  - f(sum)=sum
  - between $-\infty$ and $+\infty$
  - no transformation



Activation function f



- **Rectified Linear Unit (ReLU)**
  - Used in convolutional neural networks > deep learning
- **Exponential**
  - f(sum) = e^{sum}
  - between 0 and $+\infty$ (positive targets)
- **Radial Basis Function** (RBF Networks)
  - Gaussian activation functions
  - Lots of theory but not many practical data mining applications
- Other activation functions: arctan, Elliot, …

- Practical advice:
  - **For classification**: use tanh in hidden and logistic in output layer
  - **For regression:** use tanh in hidden and linear/tanh in output layer

3. Training

Conceptual approach

Practicalities

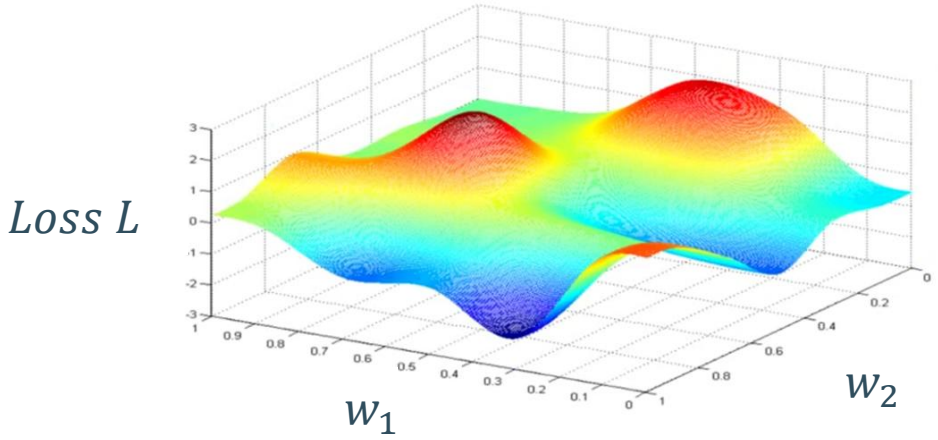
Or, if you're a statistician: fitting!



Conceptual approach

KU LEUVEN

Training



- Training an ANN = optimizing the weights
- **Loss function *L***
  - **= objective function, error function**
  - Evaluates the difference between target and prediction
  - Defines **a surface in weight space (= parameter space)**

$$y = w_2 \cdot f(w_1 \cdot x) + \varepsilon$$



- For (some) *simple* statistical models (e.g., linear regression), **closed-form analytical solutions** exist for the **optimum parameter estimates**
- For *nonlinear models* (e.g., neural networks), the parameter estimates need to be determined **numerically**, using an **iterative optimization** algorithm

Training

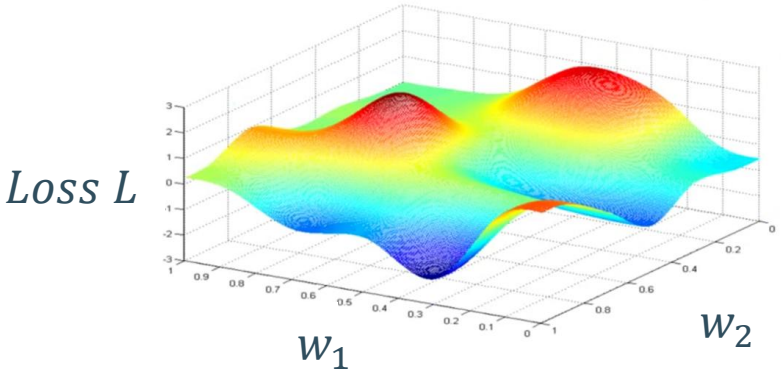
FYI: CE is equivalent with Maximum Likelihood

- **Loss function for regression**
  - **Mean squared error** (MSE): $$E = \frac{1}{2}\sum_{i=1}^{N}(y(\mathbf{x}_i;\mathbf{w}) - t_i)^2$$

    with $y(\mathbf{x}_i;\mathbf{w})$ the output of the network for observation i and t_i the true output value (target)
- **Loss function for classification**
  - **Cross-entropy** (CE): $$E = -\sum_{i=1}^{N}\sum_{k=1}^{c}(t_{ik}\ \ln(y_{ik}(\mathbf{x}_i;\mathbf{w}))$$

    with t_{ik} a 0/1 variable indicating class membership and y_{ik} the output of the network for observation i and output class k (1-of-c coding)
  - **Cross-entropy** for **binary classification**: $$E = -\sum_{i=1}^{N}(t_i\ \ln(y(\mathbf{x}_i;\mathbf{w})) + (1-t_i)\ln(1-y(\mathbf{x}_i;\mathbf{w})))$$

Training

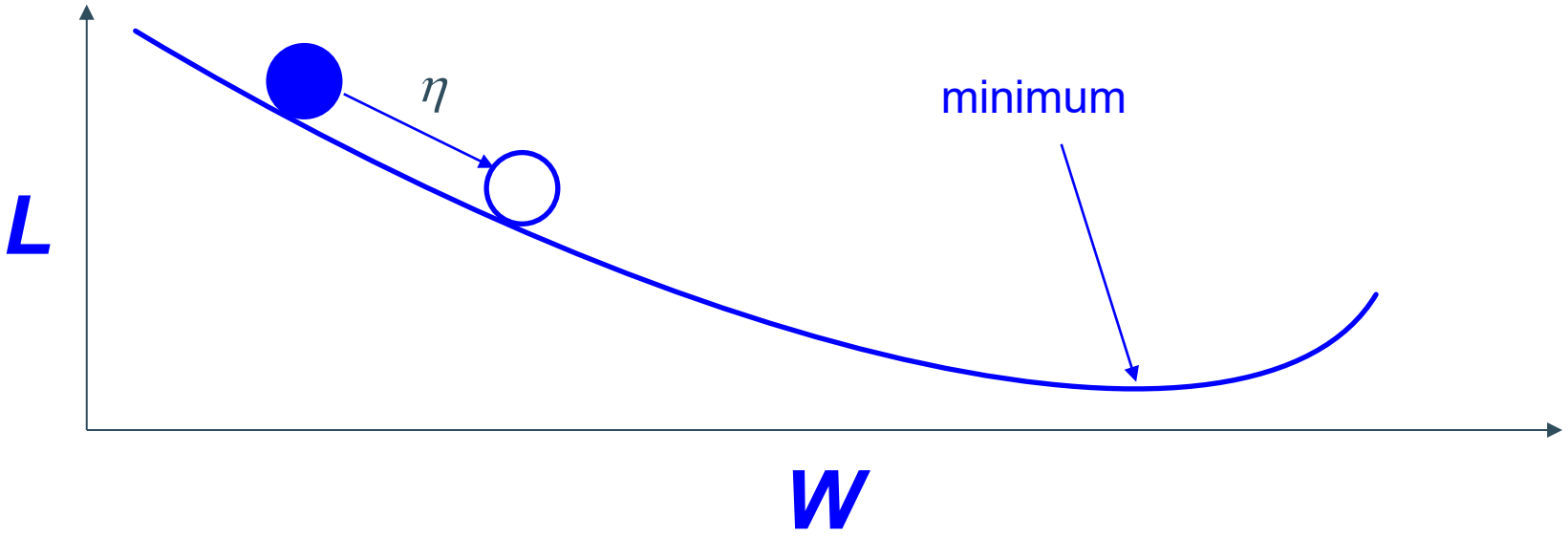


- Objective in training = finding the **optimal set of weights**
  - Optimal: optimizing **performance**, i.e., minimizing the **loss**
  - Loss function ***defines*** a **surface in parameter (weights) space**
  - For every set of weights w, the loss function returns the loss value
- **How? → Gradient Descent Learning!**
  - We don't have analytical formula describing this function, **no analytical solution**!
    - We have a definition of *L* in terms of y and ŷ, but not in terms of *w*
  - We can evaluate/approximate the value of *L* for specific values of the parameters *w*
    - Using the data sample at hand
  - We can use the **characteristics of the loss surface:** the **gradients**
    - Gradients describe **how the loss changes for (small) changes in the weights**
    - Gradients can be calculated for a specific set of weights w and the data sample at hand

Training

How to evaluate -$\delta L / \delta W$?

- **Gradient descent learning**
  - The Loss ***L*** is a **function** of the **weights** and the **training data**
  - Minimize Loss ***L*** using **gradient descent**



Downhill = - $\delta L / \delta W$

$W^{new} = W^{old} - \eta \, \delta L / \delta W^{old}$

η: *learning rate*

Training

- Preliminary step: **initialize the weights** (to small values)
- **4 steps** that are **iterated** until **convergence** is achieved



1. Forward pass

Feed the input through the network to get a **prediction**.



2. Measure loss

Compare the prediction with the target in terms of the **loss function**.



3. Backward pass

Ask every weight: how much did you **contribute** to the loss?



4. Update

Nudge every weight a little in the direction that lowers the loss.

- **Backpropagation**: practical implementation of gradient descent learning
  - Step 3, but often used to refer to the entire training process (‘backprop’)
  - Efficient way to compute all “contributions” (gradients) of weights with the chain rule
  - Most difficult step to understand conceptually (cf. infra, “backprop at work”)

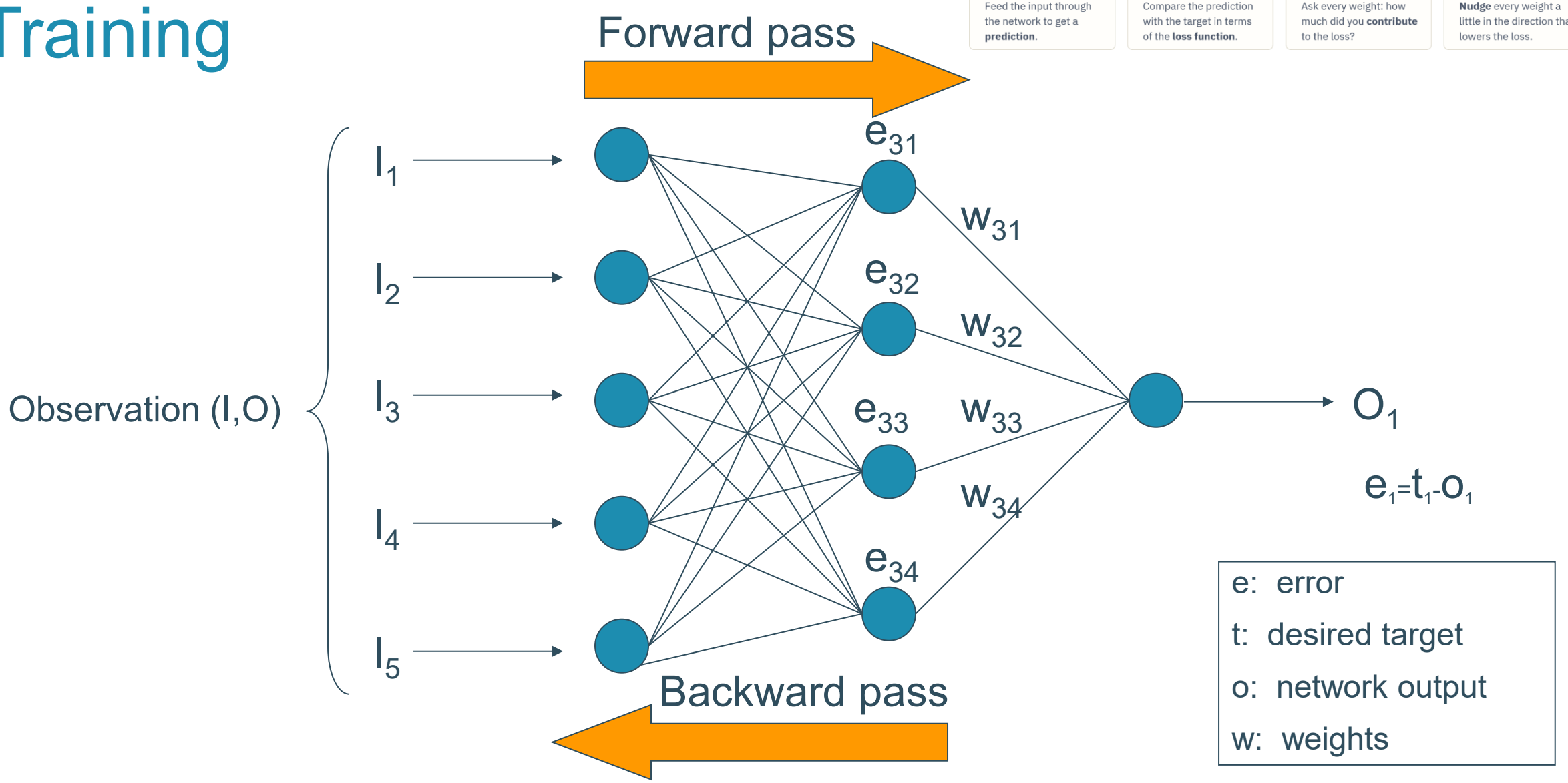
Training

1. Forward pass
Feed the input through the network to get a **prediction**.

2. Measure loss
Compare the prediction with the target in terms of the **loss function**.

3. Backward pass
Ask every weight: how much did you **contribute** to the loss?

4. Update
Nudge every weight a little in the direction that lowers the loss.



Practicalities

Data preprocessing

Preliminary training

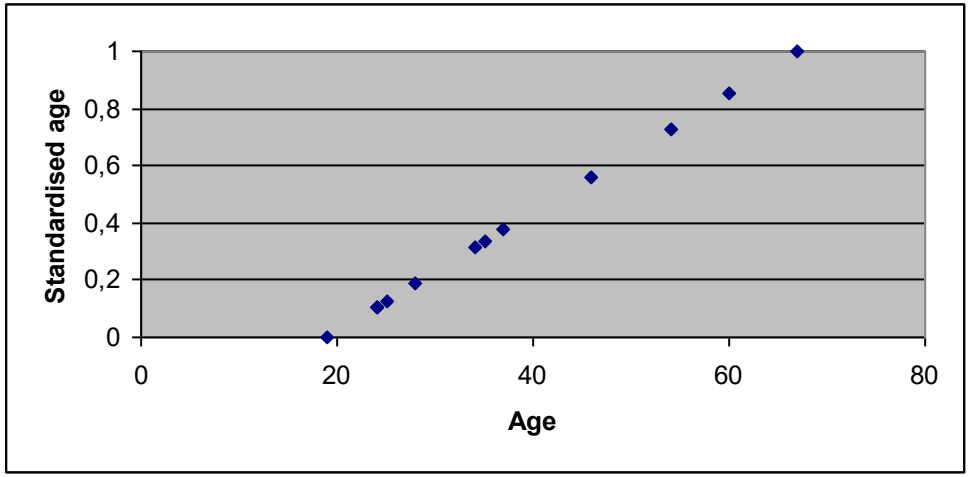
Tuning hyperparameters

Convergence criteria

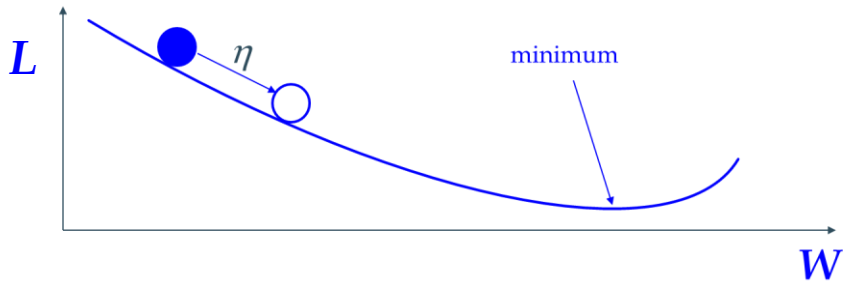
Data preprocessing

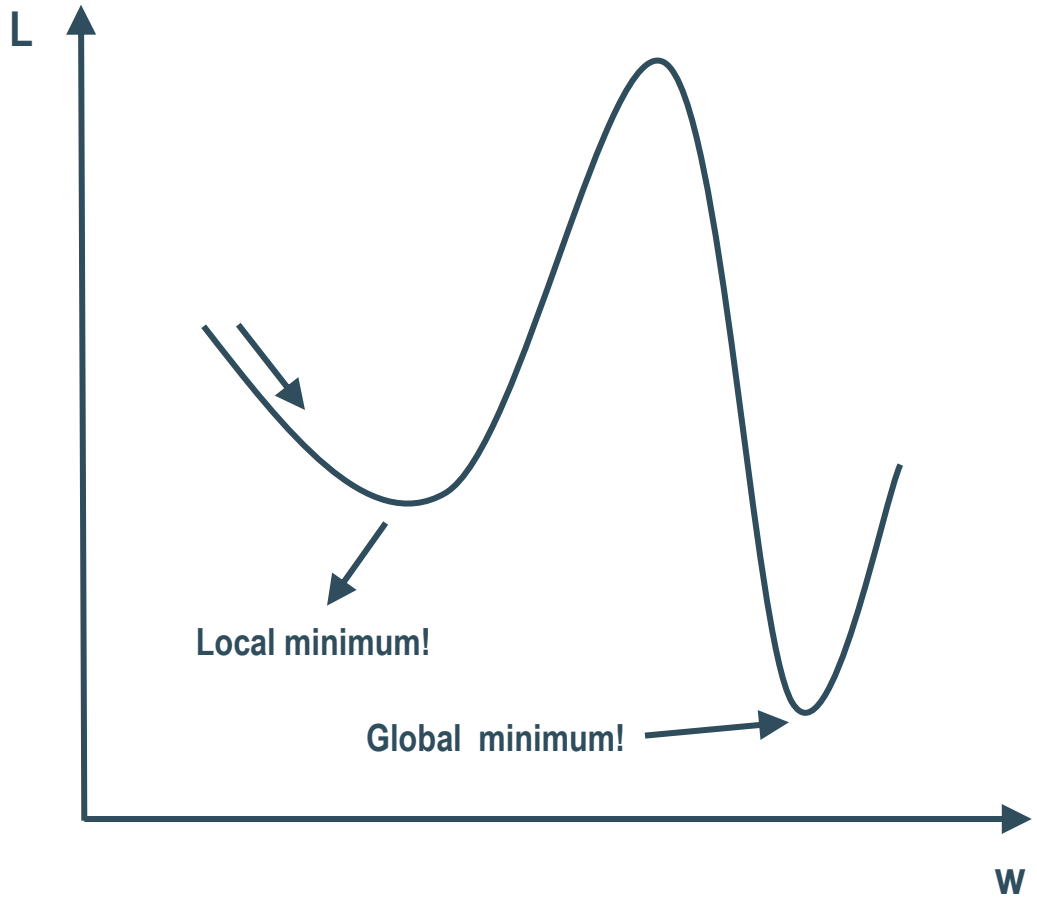
- Learning procedure may be sensitive to the scale of the inputs

- Continuous variables: **standardize**
  - To the range [0;1], [-1;1] using a linear transformation
  - Use the z-scores: $$z_i = \frac{x_i - \overline{X}}{s_X}$$
- Categorical variables
  - Use 0/1 dummy coding
  - Use weights of evidence (WOE) coding



Preliminary training





Loss function may be non-convex!

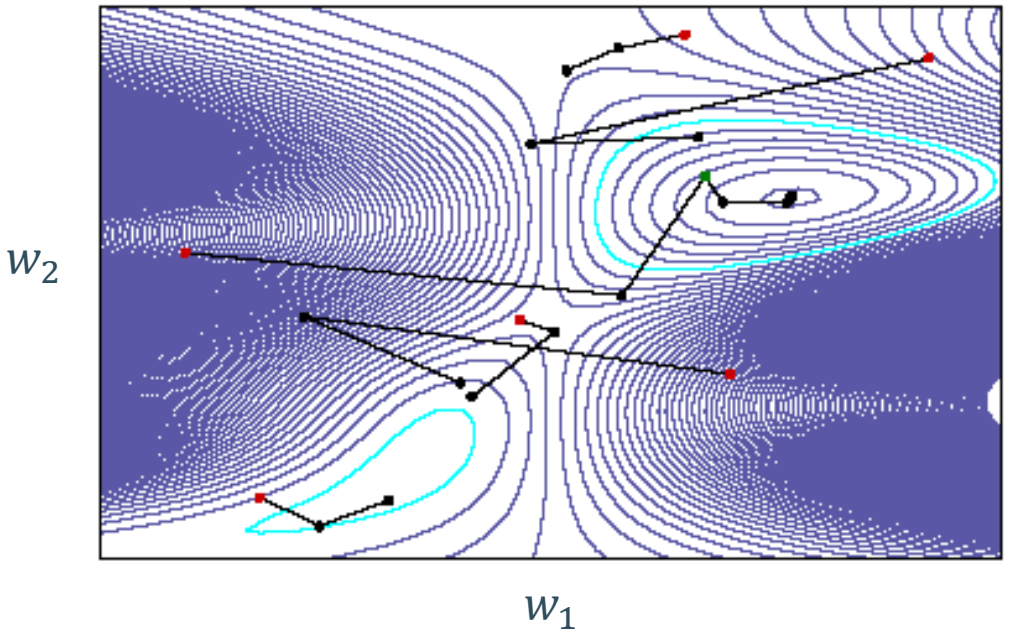
Possible solution: preliminary training

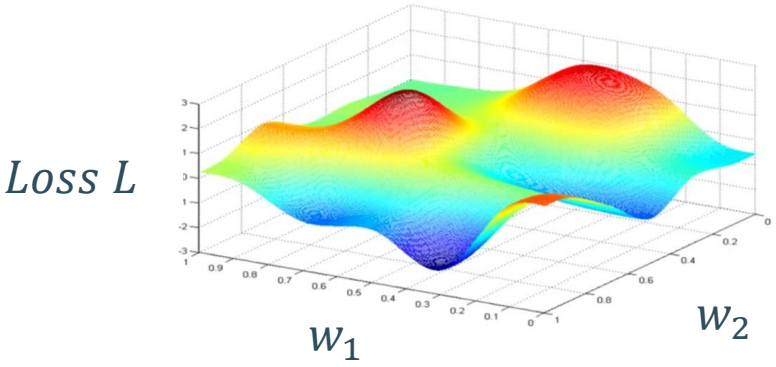
Preliminary training

- Good starting values for the weights are essential for getting good solutions (and for not getting stuck in local minima)
- **Preliminary training** uses a small number of random starting weights and takes a few iterations (20 by default) from each
- Use the best of the final values as the new starting values

Contour line plot

- Starting weights
- Updated weights
- Best weights after pretraining





Tuning hyperparameters

- **Learning rate η**
  - Too high: oscillations, algorithm will diverge
  - Too small: slow progress
  - Adaptive learning rate
- **Momentum parameter α**
  - $0 \leq \alpha \leq 1$
  - Current step direction is smoothed with the previous step
- **Architecture**: series of hyperparameters (# layers, # hidden neurons, …)

- **Extensive tuning required to optimize!**

Tuning hyperparameters

How to optimize the architecture of a neural network?
E.g., the number of neurons in a hidden layer?

1. Split data into a training set, validation set, and test set (or use cross-validation setup)
2. Vary the number of hidden neurons (and layers, and values of hyperparameters)
   - Using a predefined step size (e.g., vary neurons from 1 to 10 with stepsize 1, …)
3. Train ANNs on the training set and evaluate their performance on the validation set
   - Try out a number of ANNs each time to avoid bad local minima (cf. supra)
4. Choose the number of hidden neurons (and other hyperparameters) with optimal (average) validation set performance
5. Train a 'final' NN on training + validation set and measure final performance on the independent test set

- Specialized software support: https://wandb.ai/site, https://optuna.org/, …

Convergence criteria

= when do we stop updating the weights?

- Preliminary step: **initialize the weights** (to small values)
- **4 steps** that are **iterated** until **convergence** is achieved

- Loss function shows no progress
- The weight parameter estimates stop changing substantially
- The gradient is close to zero
- Use a validation set!
- Fixed number of epochs

4. Learning vs. Overfitting

Early stopping

Regularization

Input selection with pruning

Input selection with wrapper

Learning versus Overfitting

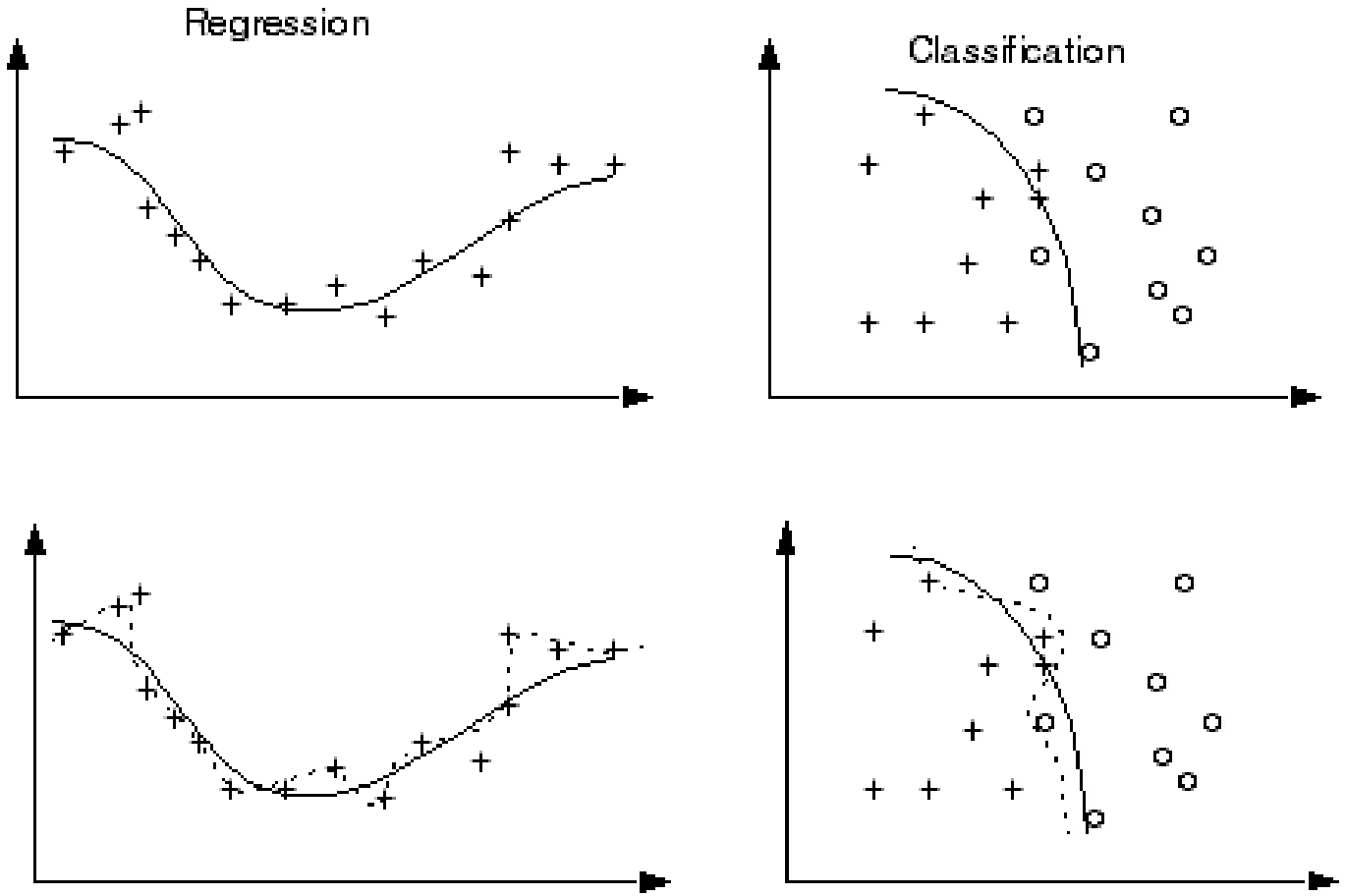
- **Successful learning**:
  - Successfully recognize data outside the training set, that is, data in an independent test set (i.e., generalize well)

- **Overfitting** (‘memorization’)
  - Each data set is characterized by noise (idiosyncrasies) due to incorrect entries, human errors, irrationalities, noisy sensors, ….
  - A model that is **too complex** (for example, decision tree with too many nodes, neural network with too many hidden neurons) **might fit the noise**, not just the signal, **leading to overfitting**

Learning versus Overfitting

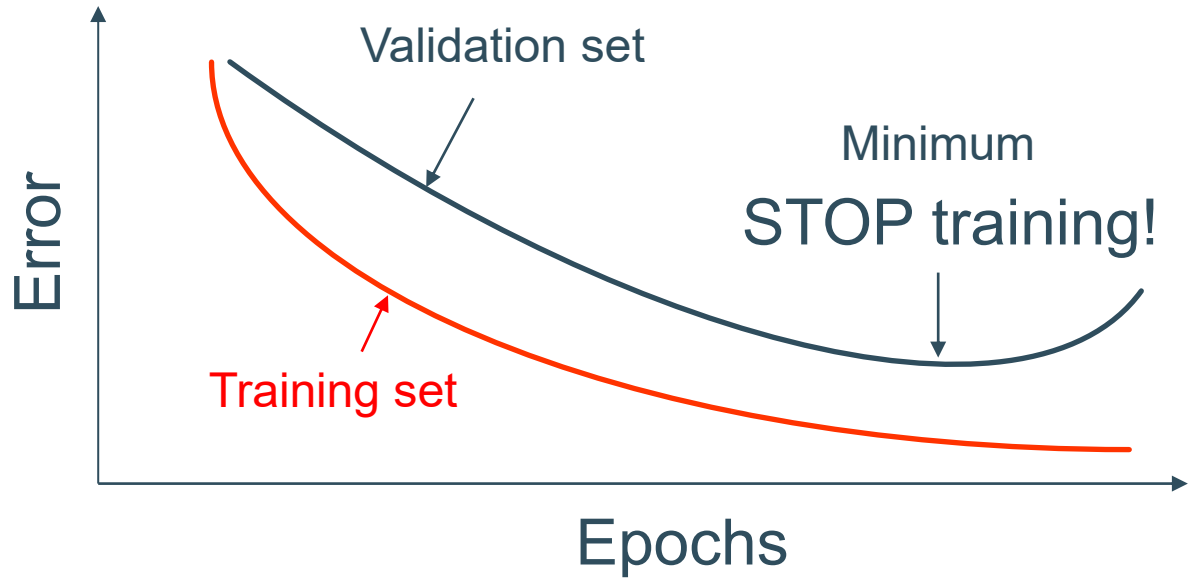
-----: Overfitting

——: 'True' model

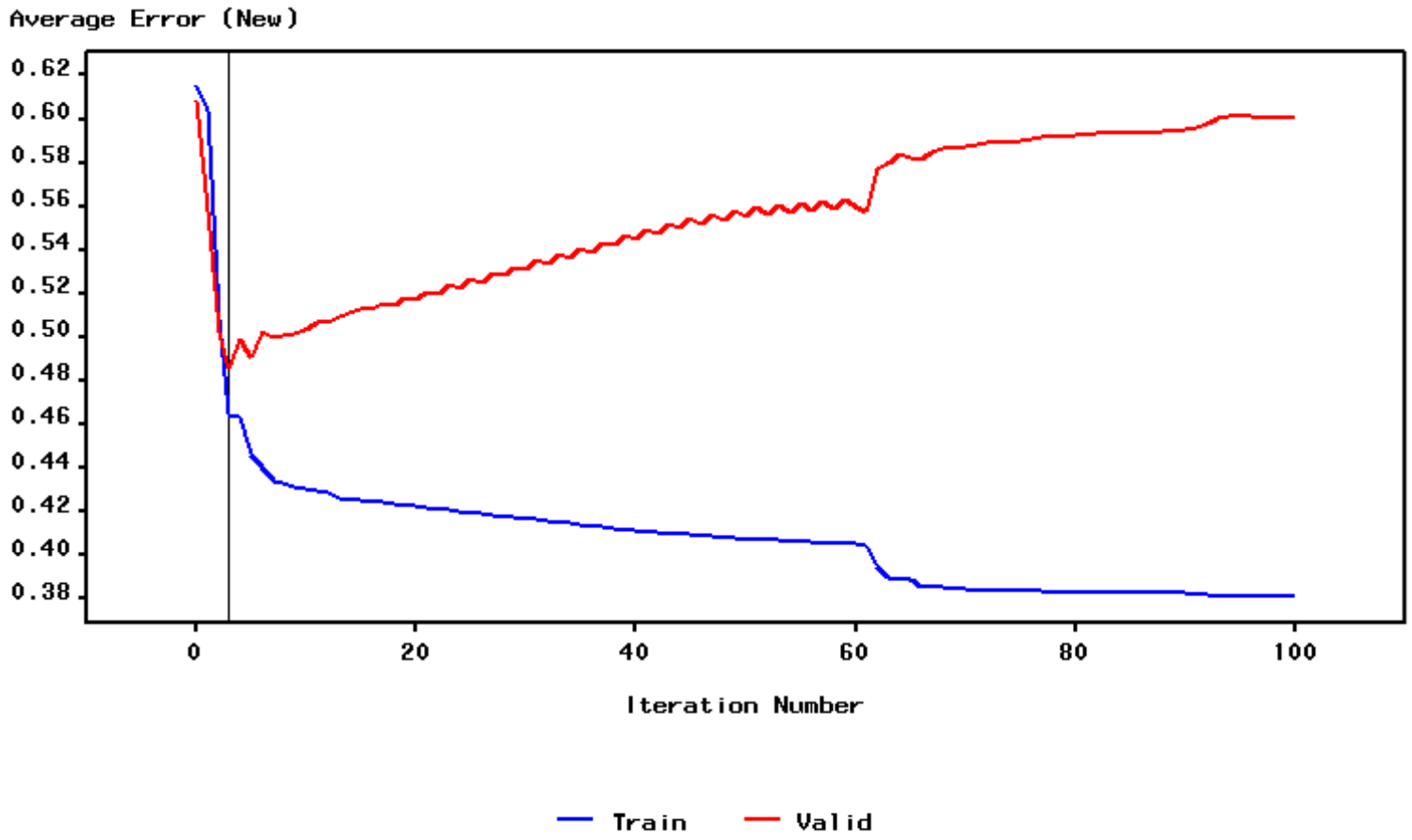


Avoiding overfitting: early stopping

- Set aside validation set
  - E.g., 30% of observations
- Use training set to estimate weights
- Use validation set to decide when to stop updating the weights
  - When overfitting occurs ('early stopping')
  - Or: postpruning (cf. decision trees)
- Evaluation metric can be different from objective function (e.g., AUC, profit-based, …)
- Start from **oversized network** with *enough* hidden neurons



Avoiding overfitting: early stopping



Avoiding overfitting: regularization

- Bartlett (1997) demonstrated that **generalization depends more on the size of the weights than the number of weights**
  - A large network with small weights acts like a smaller, less complex network
- Thus, **restraining the weights** should prevent a bumpy overfitted surface
  - However, it might also prevent the model from adapting to true features of the data

Bias – Variance trade-off (cf. Chapter 4)

- **Penalize large weights (in absolute sense) in the objective function!**

Avoiding overfitting: regularization

$$\text{Objective Function} = \text{Loss Function} + \lambda \|w\|^2 = \text{Loss Function} + \lambda \sum_{i=1}^{n} w_i^2$$

- The decay (shrinkage, smoothing) parameter λ controls the severity of the penalty
- Trade-off:
  - Setting λ too low might cause overfitting
  - Setting λ too high might cause underfitting

Bias – Variance trade-off!

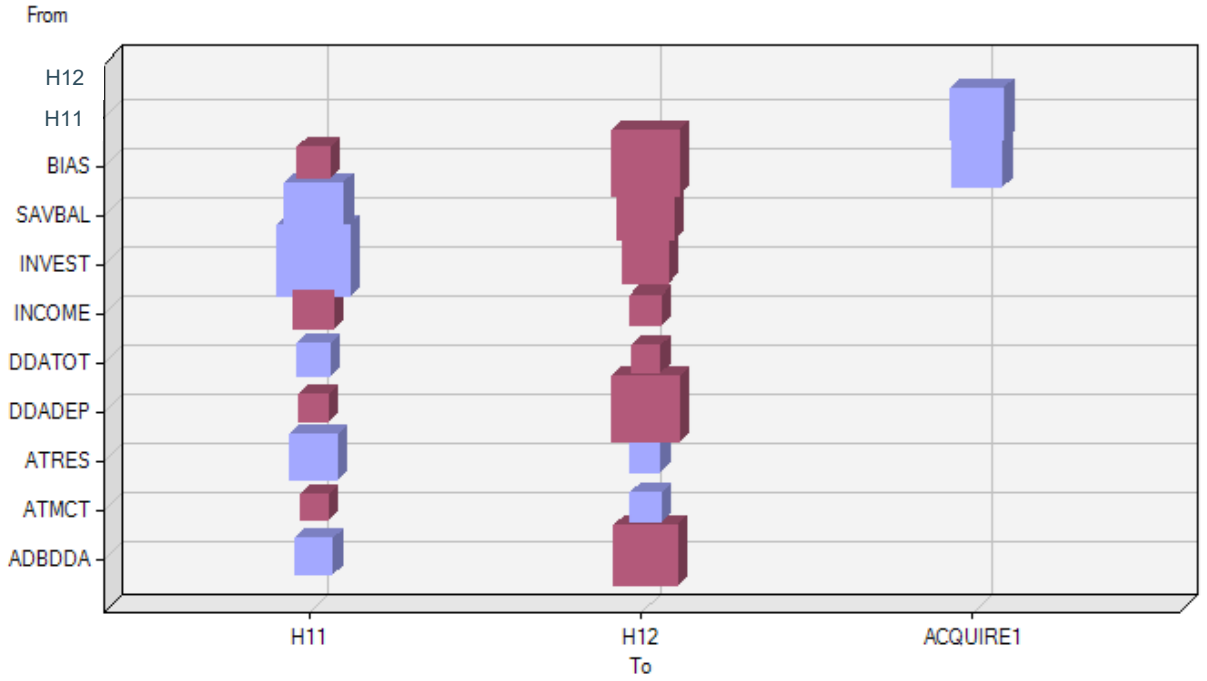
- **Tune** λ using a separate **validation set**
- Same principle used by **Ridge (L2)** and **Lasso (L1) regression** (& Elastic nets)
  - Extensions to linear or logistic regression that adopt regularization

Avoiding overfitting: input selection with **pruning**

- Input selection = variable selection (cf. regression)
  - Complicated in neural networks because each input is associated with h input-to-hidden layer weights and h hidden-to-output layer weights
  - However, if all input-to-hidden layer weights for an input are zero (or close), then the input is clearly not important
- **Pruning algorithm:**
  - Train the neural network
  - Prune the input where the input-to-hidden layer weights are closest to zero
  - Retrain the network (might be starting from previous weights to speed up convergence)
  - If the predictive power increases (or stays the same), then repeat.
  - If not, reconnect the input and stop

Avoiding overfitting: input selection with **pruning**

Hinton diagrams:



Two hidden neurons:
(H11, H12)

One output:
(ACQUIRE1)

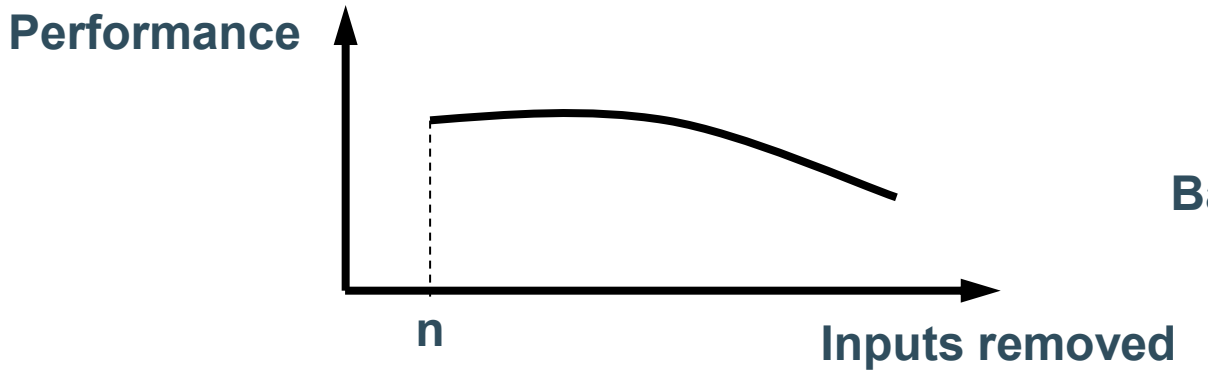
Geoffrey Hinton:
https://www.nytimes.com/2019/03/27/technology/turing-award-ai.html
https://www.youtube.com/watch?v=-cv0ddgcImk
https://www.nobelprize.org/prizes/physics/2024/press-release/

Size of square indicates size of weight
Color of square indicates sign of weight

Avoiding overfitting: input selection with **wrapper**

Input selection with a wrapper approach

1. Start from a neural network with all **n** inputs.
2. Remove each input in turn and re-estimate the network.
3. Exclude the input whose absence gives the best performing network (for example, in terms of AUC, MSE, …).
4. Repeat this procedure until performance decreases significantly



Backward or forward!

5. Demo

Backprop at work

Neural network playground

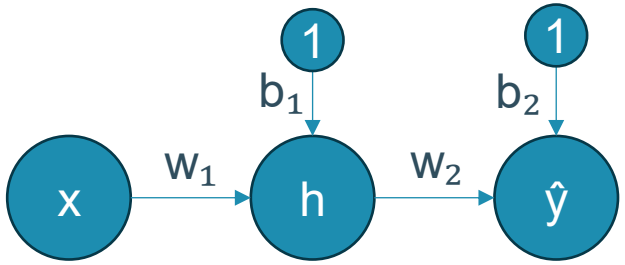
For illustration purposes
No need to be able to calculate gradients!
You do need to understand conceptually!

Backprop at work

Calculations involved in training a simple network

KU LEUVEN

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

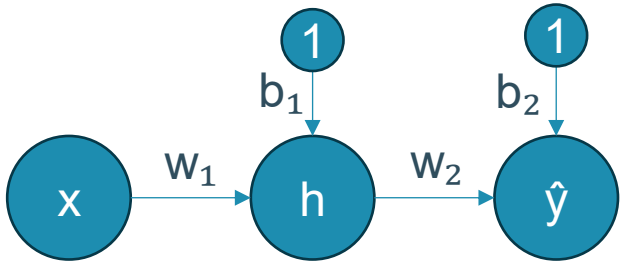
- **Simple network**:
  - One input x
  - One hidden neuron, with a **sigmoid activation**
  - One output neuron, with a **linear activation**
  - Hidden and output neuron have their own **bias**

$\sigma(z_1) = 1 / (1 + e^{(-z_1)})$

$\hat{y} = z_2 = w_2 \cdot x + b_2$

Step	Formula
Hidden pre-activation	$z_1 = w_1 \cdot x + b_1$
Hidden activation	$h = \sigma(z_1) = 1 / (1 + e^{(-z_1)})$
Output (linear)	$\hat{y} = w_2 \cdot h + b_2$

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

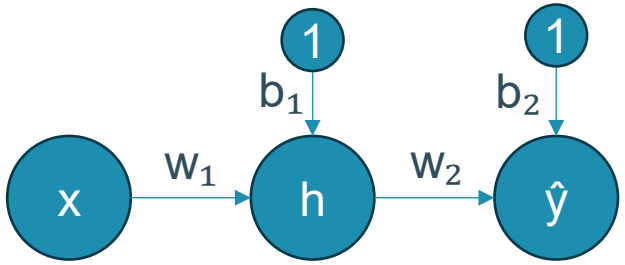
$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Network initialization**
  - $w_1 = 0.5$
  - $b_1 = -1$
  - $w_2 = 1$
  - $b_2 = 0$
  - Learning rate $\eta = 0.5$

All numbers in this example have been chosen for ease of calculations (cf. infra)

Step	Formula
Hidden pre-activation	$z_1 = w_1 \cdot x + b_1$
Hidden activation	$h = \sigma(z_1) = 1 / (1 + e^{(-z_1)})$
Output (linear)	$\hat{y} = w_2 \cdot h + b_2$

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

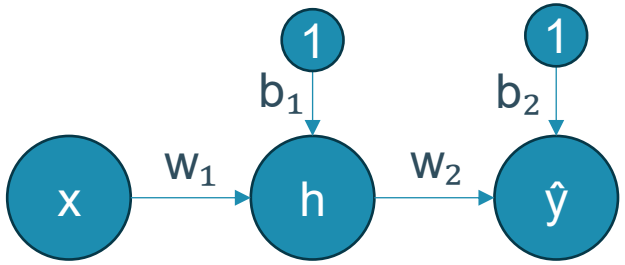
$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 1: Forward pass**
  - Training observation
    - x = 2
    - y = 1
  - Calculations using formulas

All numbers in this example have been chosen for ease of calculations (cf. infra)

Step	Formula	Calculation	Value
Hidden pre-activation	$z_1 = w_1 \cdot x + b_1$	$0.5 \cdot 2 + (-1)$	0
Hidden activation	$h = \sigma(z_1) = 1 / (1 + e^{\wedge}(-z_1))$	$\sigma(0)$	0.5
Output (linear)	$\hat{y} = w_2 \cdot h + b_2$	$1 \cdot 0.5 + 0$	**0.5**

Backprop at work



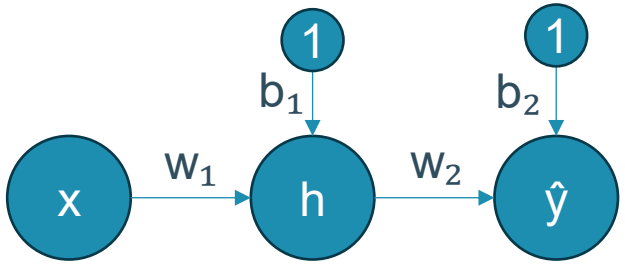
$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 2: Measure the loss**
  - Then how to update weights and biases to **reduce loss**?
    - Which knob do we turn, and which way?
  - Gradient ***∂L/∂parameter***:
    - Tells how much loss changes when nudging parameter (a tiny bit)
    - **Sign**: which way to turn the knob
    - **Size**: how much does the knob matter right now

Step	Formula	Calculation	Value
Hidden pre-activation	$z_1 = w_1 \cdot x + b_1$	$0.5 \cdot 2 + (-1)$	0
Hidden activation	$h = \sigma(z_1) = 1 / (1 + e^{(-z_1)})$	$\sigma(0)$	0.5
Output (linear)	$\hat{y} = w_2 \cdot h + b_2$	$1 \cdot 0.5 + 0$	**0.5**
Loss (squared error)	$L = ½ (\hat{y} - y)^2$	$½(0.5 - 1)^2$	0.125

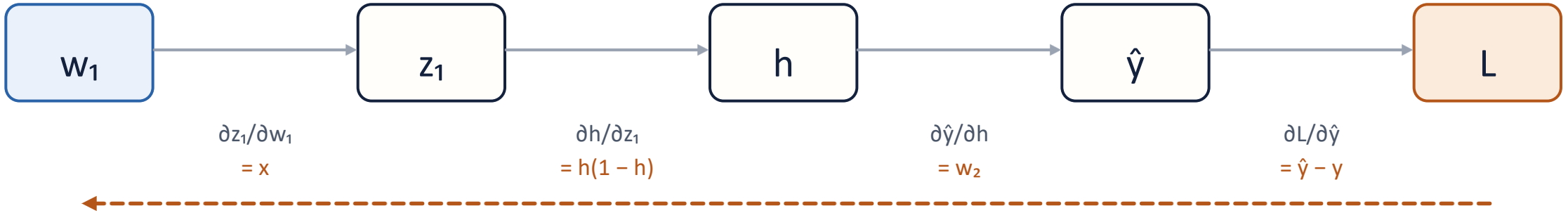
Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

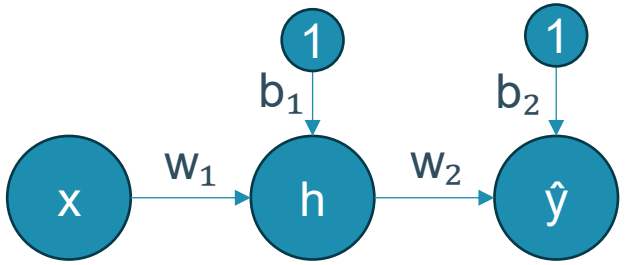
- **Step 3: Backward pass**
  - The **chain rule**: multiply *local gradients*
    - E.g.: if we want to know how the Loss changes when we nudge weight w_1



$$\partial L/\partial w_1 = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial h \cdot \partial h/\partial z_1 \cdot \partial z_1/\partial w_1$$

$$\partial L/\partial w_1 = (\hat{y} - y) \cdot w_2 \cdot h(1 - h) \cdot x$$

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

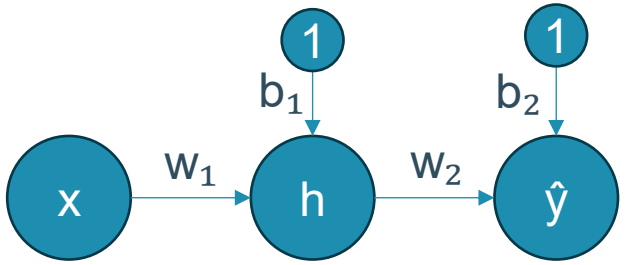
$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 3: Gradients**
  - Derivatives needed for calculating the gradients of the loss

Derivative	Expression
$\partial L/\partial \hat{y}$	$= \hat{y} - y$ (clean 'error signal' explaining use of MSE with ½ as (differentiable) Loss)
$\partial \hat{y}/\partial w_2$	$= h$
$\partial \hat{y}/\partial b_2$	$= 1$
$\partial \hat{y}/\partial h$	$= w_2$
$\partial h/\partial z_1$	$= \partial\sigma(z_1)/\partial z_1 = \sigma(z_1)\cdot(1 - \sigma(z_1)) = h(1 - h)$
$\partial z_1/\partial w_1$	$= x$
$\partial z_1/\partial b_1$	$= 1$

Note: The ½ in the loss ($L = ½ (\hat{y} - y)^2$) is a *convenience*: it cancels when we take the derivative of L

Backprop at work



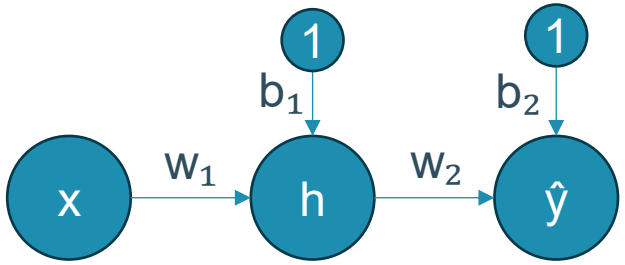
$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 3: Gradients**
  - Using the chain rule to evaluate the gradient of the loss with respect to:
    - Parameters at output layer: w_2 and b_2
    - Parameters at hidden layer: w_1 and b_1

Derivative	Chain rule	Gradient
$\partial L/\partial w_2$	$\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial w_2$	$(\hat{y} - y) \cdot h$
$\partial L/\partial b_2$	$\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial b_2$	$(\hat{y} - y) \cdot 1$
$\partial L/\partial w_1$	$\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial h \cdot \partial h/\partial z_1 \cdot \partial z_1/\partial w_1$	$(\hat{y} - y) \cdot w_2 \cdot h(1 - h) \cdot x$
$\partial L/\partial b_1$	$\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial h \cdot \partial h/\partial z_1 \cdot \partial z_1/\partial b_1$	$(\hat{y} - y) \cdot w_2 \cdot h(1 - h) \cdot 1$

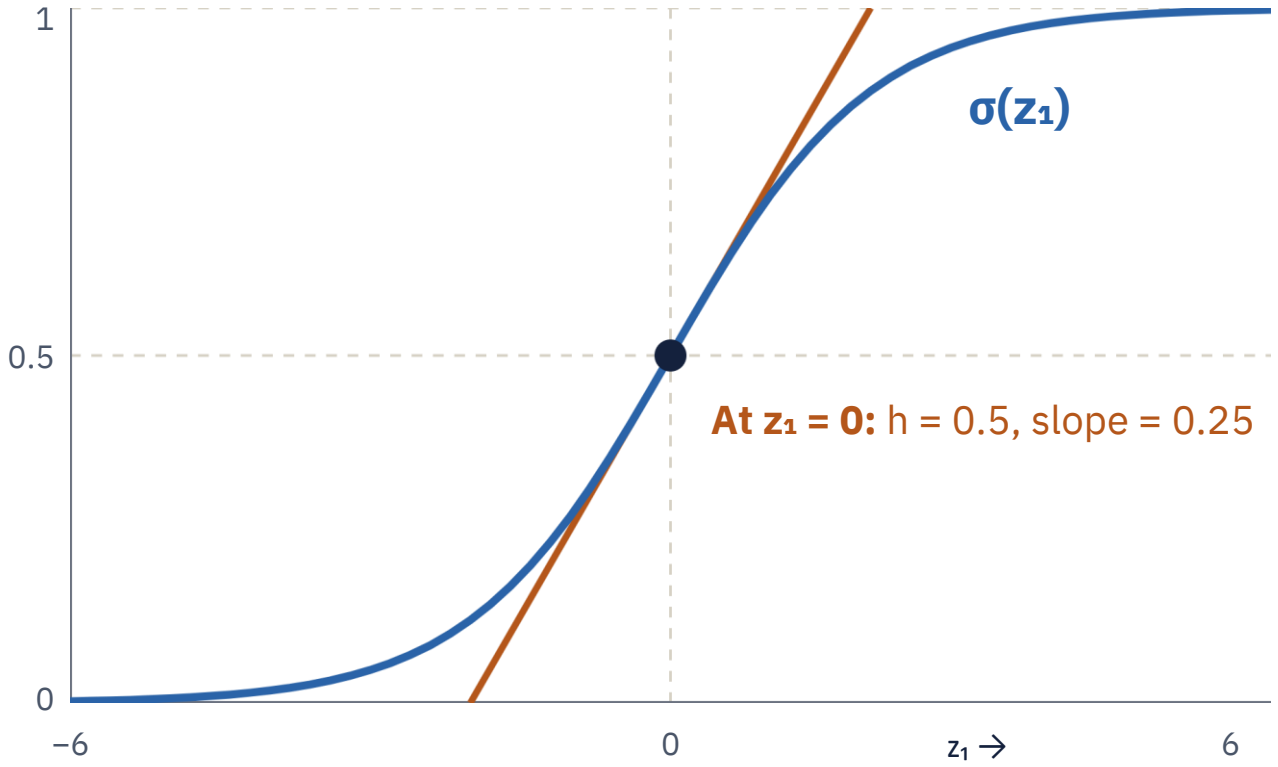
Backprop at work



$$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$$

$$\hat{y} = z_2 = w_2 \cdot h + b_2$$

- **Step 3: Gradients**
  - The sigmoid and its slope



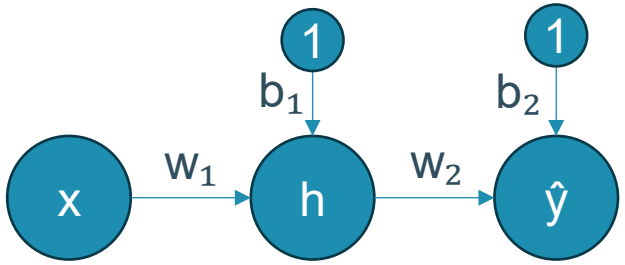
The sigmoid squashes any number into a value between 0 and 1.

$$\sigma(z) = 1/(1 + e^{-z})$$

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

At our point: $\sigma' = 0.5 \times 0.5 = \mathbf{0.25}$. The slope says how strongly a nudge in z_1 moves h. The backward pass needs exactly this number.

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 3: Gradients at the output layer**
  - Using the chain rule to evaluate the gradient with respect to w_2 and b_2

$\delta_2 = \partial L / \partial \hat{y} = \hat{y} - y = 0.5 - 1$

Error signal at the output (an intermediate result)

−0.5

$\partial L / \partial w_2 = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial w_2 = \delta_2 \cdot h = -0.5 \times 0.5$

Gradient for w_2: it matters in proportion to its input h

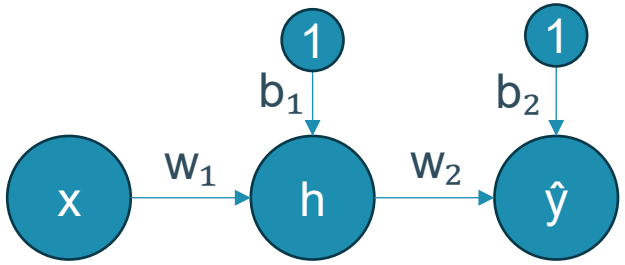
−0.25

$\partial L / \partial b_2 = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial b_2 = \delta_2 \cdot 1$

Gradient for b_2: a bias always sees an input of 1

−0.5

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

- **Step 3: Gradients at the hidden layer**

$\partial L/\partial h = \delta_2 \cdot w_2 = -0.5 \times 1$

The error travels back through the weight w_2

−0.5

$\delta_1 = \partial L/\partial h \cdot h(1 - h) = -0.5 \times 0.25$

...and through the slope of the sigmoid: the hidden neuron's error signal

−0.125

$\partial L/\partial w_1 = \delta_1 \cdot x = -0.125 \times 2$

Gradient for w_1: it mattered in proportion to its input x

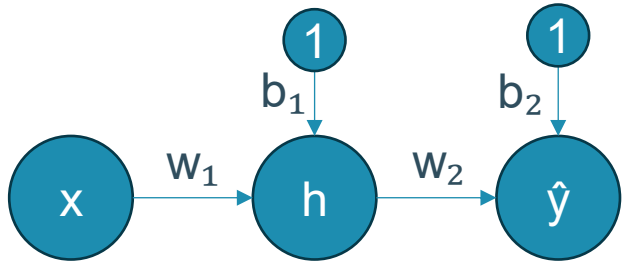
−0.25

$\partial L/\partial b_1 = \delta_1 \cdot 1$

Gradient for b_1: again, a bias sees an input of 1

−0.125

Backprop at work



$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$

$\hat{y} = z_2 = w_2 \cdot h + b_2$

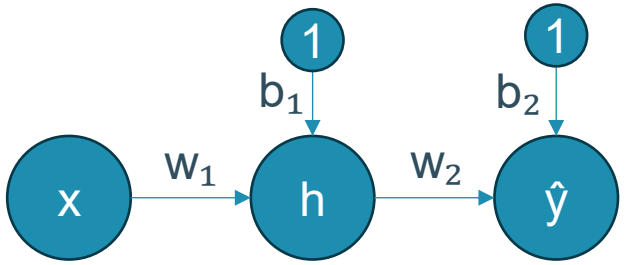
- **Step 4: Update parameters**

new = old − η × gradient with learning rate η = 0.5

Parameter	Old	Gradient	− η × grad.	New
w_1	0.5	−0.25	+0.125	0.625
b_1	−1	−0.125	+0.0625	−0.9375
w_2	1	−0.25	+0.125	1.125
b_2	0	−0.5	+0.25	0.25

All four gradients are negative, so all four parameters go **up**. Each change pushes ŷ towards the target of 1.

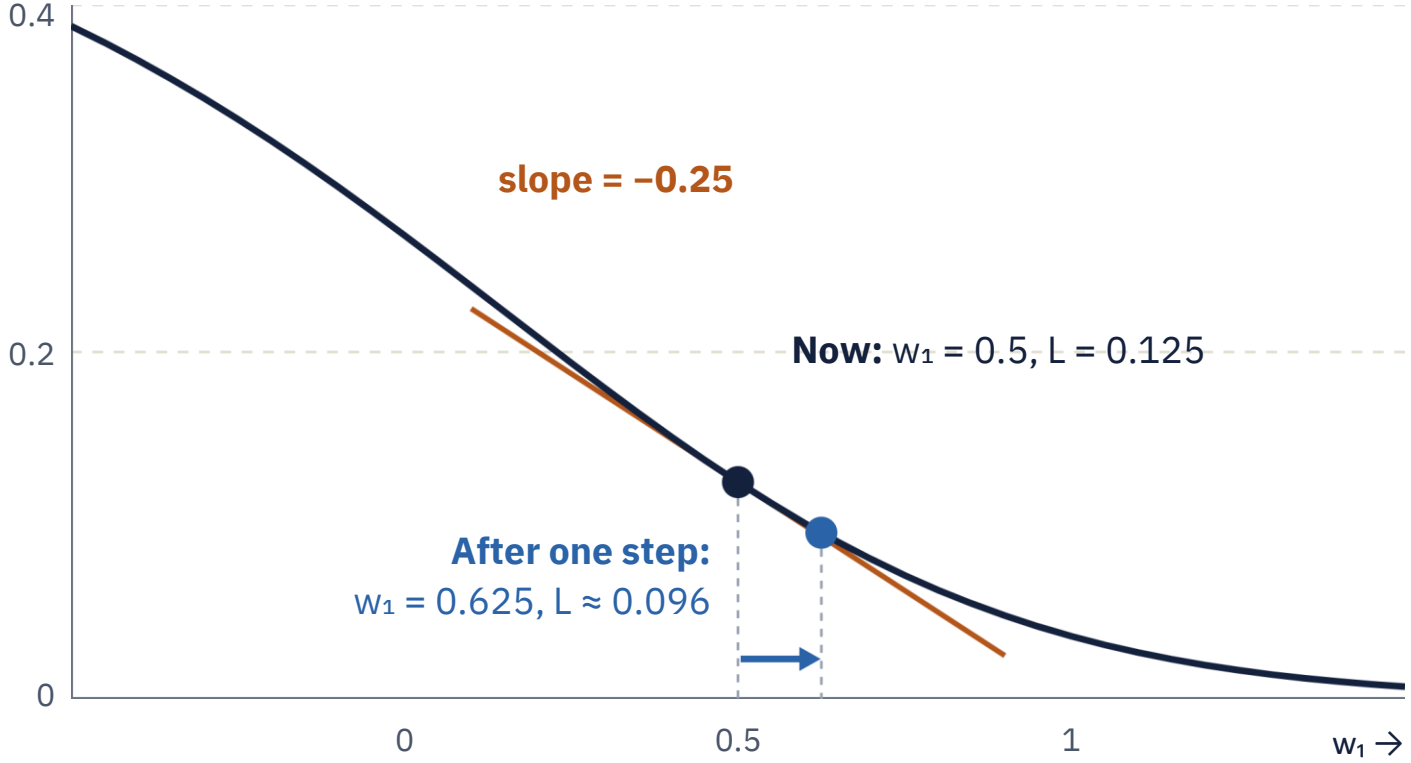
Backprop at work



$$h = \sigma(z_1) = \sigma(w_1 \cdot x + b_1)$$

$$\hat{y} = z_2 = w_2 \cdot h + b_2$$

- **Step 4: Update parameters**
  - Why minus the gradient? Walking downhill



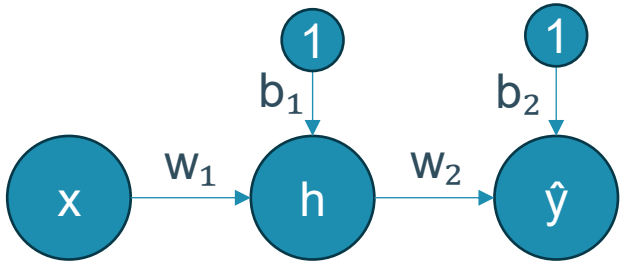
Loss as a function of w_1, other parameters fixed

The gradient is the **slope**. Here it is negative: the loss goes down to the right.

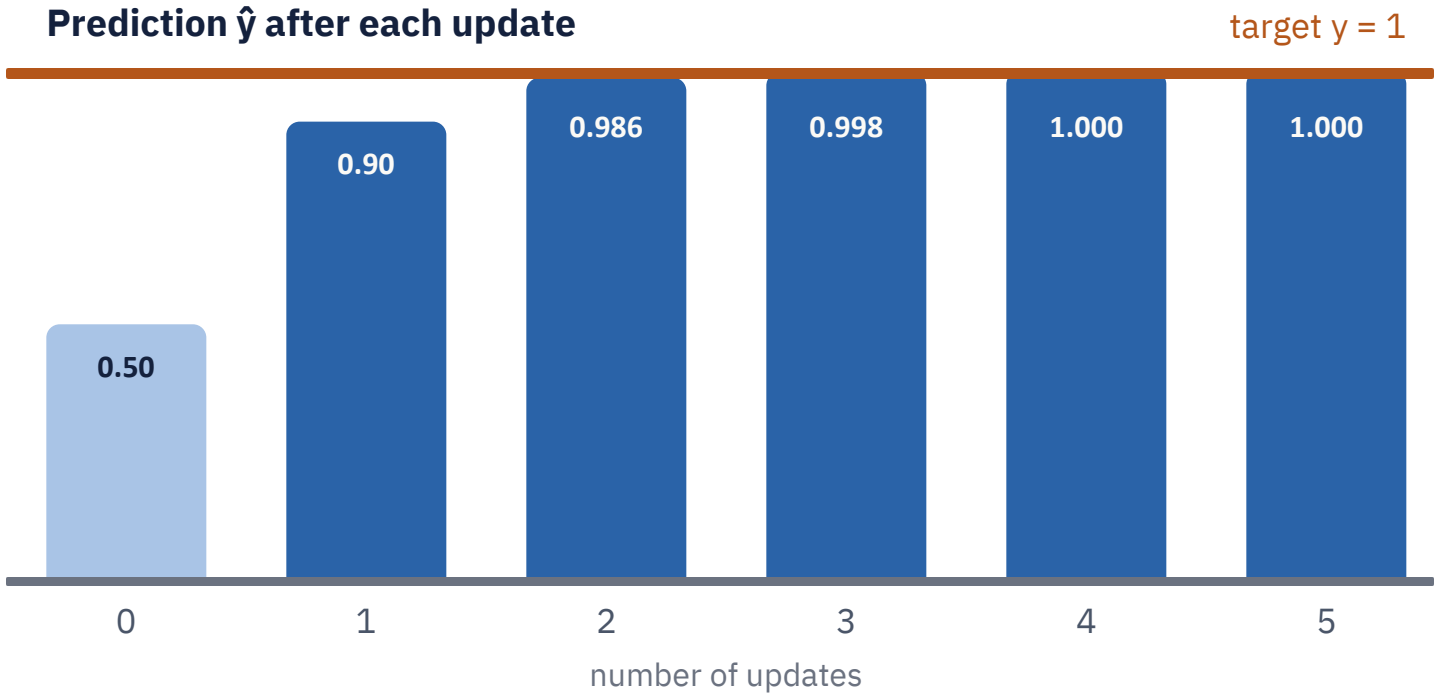
So we step right:
$-\eta \times \text{slope} = -0.5 \times (-0.25) = \mathbf{+0.125}$

Changing only w_1 lowers the loss to 0.096. Changing all four parameters together brings it to 0.005.

Backprop at work



- **Iterate step 1 > 4: forward, loss, backward, update… and repeat**

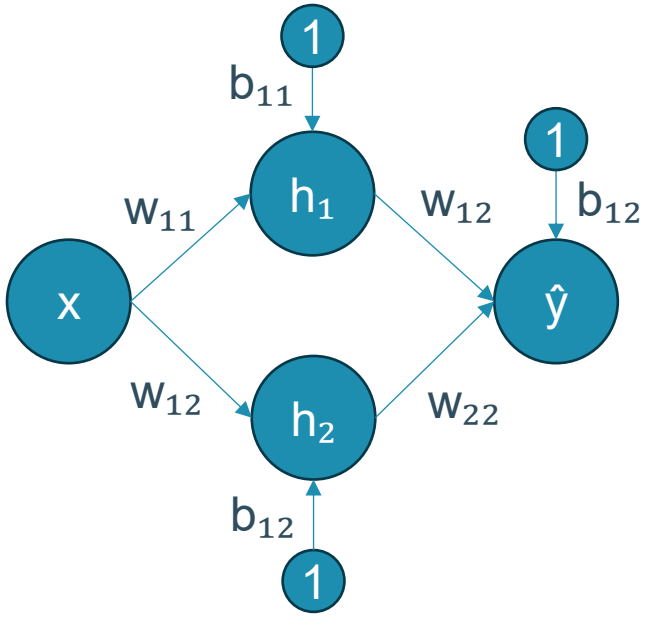


Updates	Loss
0	0.125
1	0.005
2	0.0001
3	0.000002

One training example is easy to fit perfectly. With real data the loss levels off above zero, but every step works exactly like the one we just did.

Backprop at work

- Scale-up:
  - Adding neurons
  - Adding layers



Each hidden neuron gets its own **error signal**:

δ for $h_1 = \delta_2 \times$ (weight $h_1 \rightarrow \hat{y}$) $\times h_1(1 - h_1)$

δ for $h_2 = \delta_2 \times$ (weight $h_2 \rightarrow \hat{y}$) $\times h_2(1 - h_2)$

The output error δ_2 is computed **once** and shared by both paths. That reuse is what makes backpropagation efficient.

Backprop at work

- **Online (incremental) learning**
  - Update the weights after reading each observation
- **Batch learning**: standard approach
  - Update the weights after reading the entire data set
  - Run forward and backward pass for each observation separately
  - Results in one gradient per observation per parameter
  - Average gradients across the batch
  - Do one update: $w \leftarrow w - \eta \cdot$ (average gradient)
  - 1 epoch = 1 run of all training observations through the network
- **Mini-batch learning**: for a sample of observations
  - Sample size = hyperparameter

Backprop at work

- **Batch learning**
  - Run the forward and backward pass for each observation separately
    - This results in one gradient per observation per parameter
  - Average those gradients across the batch
  - Do one update: $w \leftarrow w - \eta \cdot$ (average gradient)

	Obs. 1 (x=2, y=1)	Obs. 2 (x=1, y=0)	Batch average
Loss	0.125	0.071	0.098
$\partial L/\partial w_1$	−0.25	+0.089	−0.081
$\partial L/\partial b_1$	−0.125	+0.089	−0.018
$\partial L/\partial w_2$	−0.25	+0.143	−0.054
$\partial L/\partial b_2$	−0.5	+0.378	−0.061

Neural network playground

- https://playground.tensorflow.org/

KU LEUVEN

Q&A

- More (a bit) on Deep Learning:
  - Chapter 7
  - Chapter 9
  - …

- Serve yourselves:
  - https://udlbook.github.io/udlbook/

